

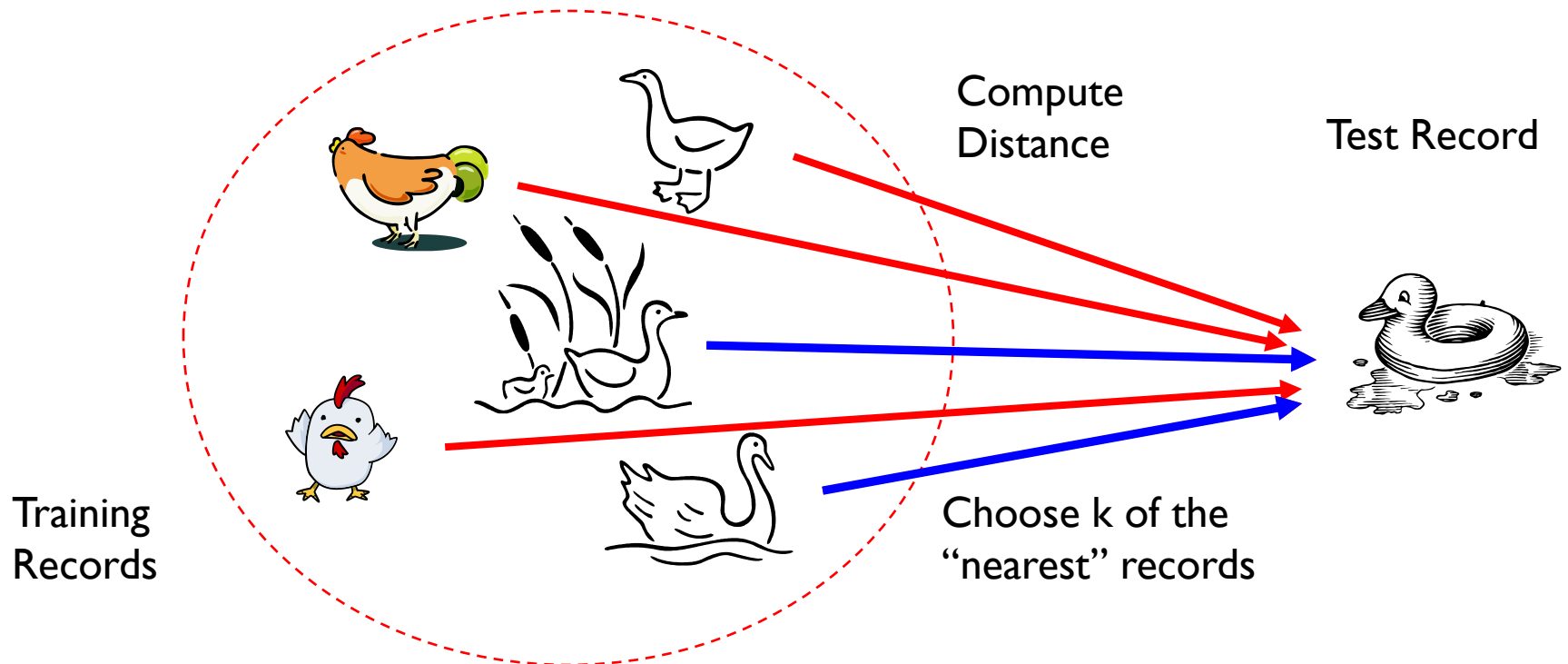
K Nearest Neighbor Classification



Nearest Neighbor Classifiers

► Basic idea:

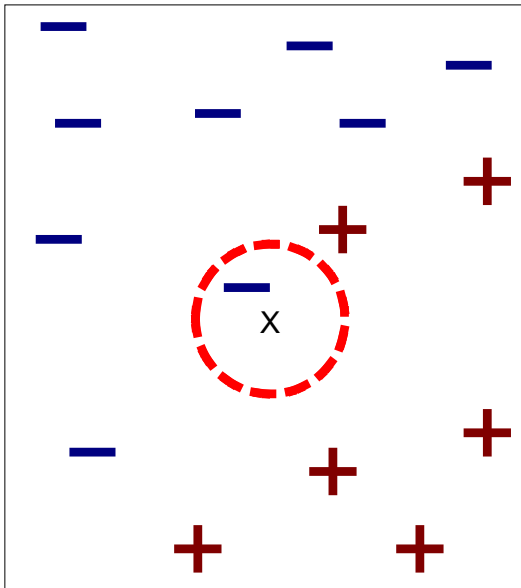
- If it walks like a duck, quacks like a duck, then it's probably a duck



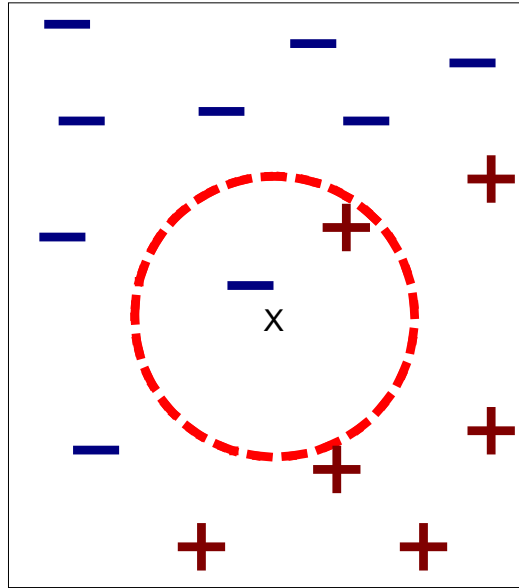
Basic Idea

- ▶ K-Nearest Neighbors (KNN) is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks. It works by finding the "k" closest data points (neighbors) to a given input and makes a predictions based on the majority class (for classification) or the average value (for regression).
- ▶ In practice, k is usually chosen to be odd, so as to avoid ties
- ▶ When $k = 1$, the algorithm simply assigns the class of the single closest data point. This is called the nearest-neighbor classification rule.

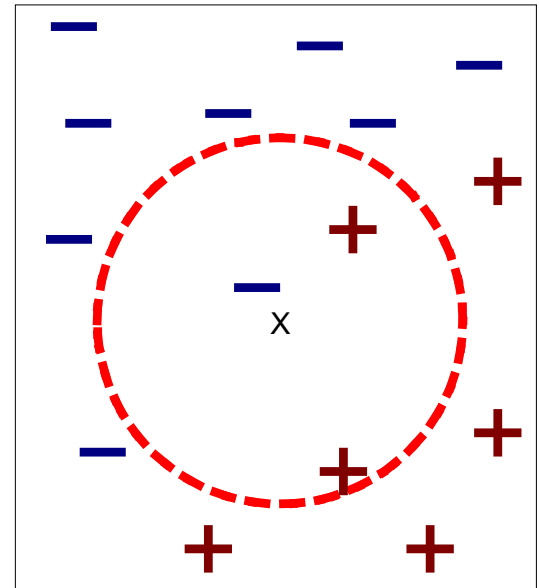
Definition of Nearest Neighbor



(a) 1-nearest neighbor



(b) 2-nearest neighbor

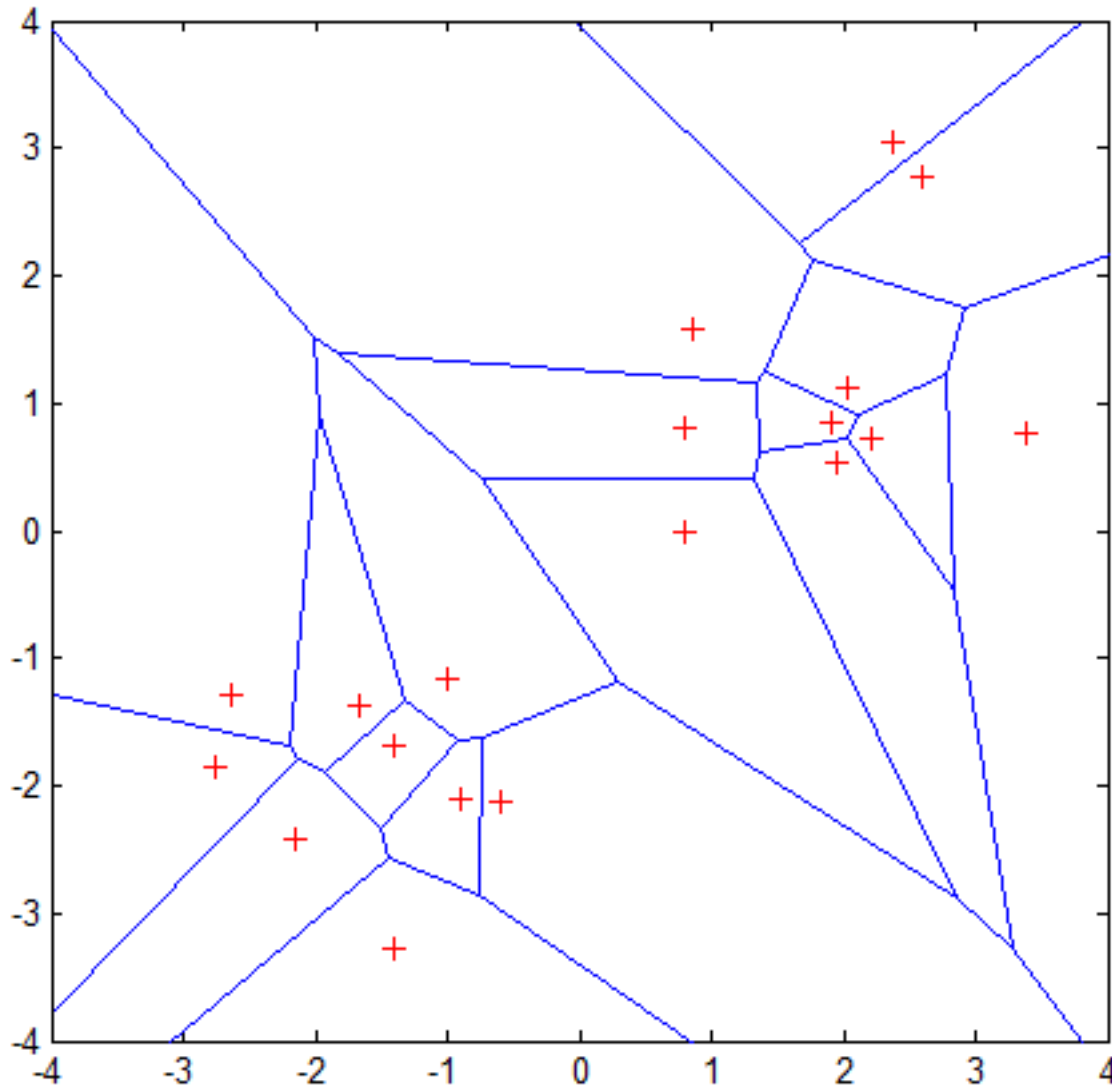


(c) 3-nearest neighbor

The k -nearest neighbors of a data point x are the k data points that are closest to x , based on the smallest distance.



Voronoi Diagram



Properties:

- 1) All possible points within a sample's Voronoi cell are the nearest neighboring points for that sample
- 2) For any sample, the nearest sample is determined by the closest Voronoi cell edge

Nearest Neighbor Classification

- Compute distance between two points:

- Euclidean distance

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

- Manhattan distance

$$d(p, q) = \sum_i |p_i - q_i|$$

- q norm distance

$$d(p, q) = (\sum_i |p_i - q_i|^q)^{1/q}$$



Nearest Neighbor Classification

- Determine the class from nearest neighbor list
 - take the majority vote of class labels among the k-nearest neighbors

$$y' = \operatorname{argmax}_v \sum_{(x_i, y_i) \in D_z} I(v = y_i)$$

where D_z is the set of k closest training examples to z.

- Weigh the vote according to distance

$$y' = \operatorname{argmax}_v \sum_{(x_i, y_i) \in D_z} w_i \times I(v = y_i)$$

- weight factor, $w = 1/d^2$



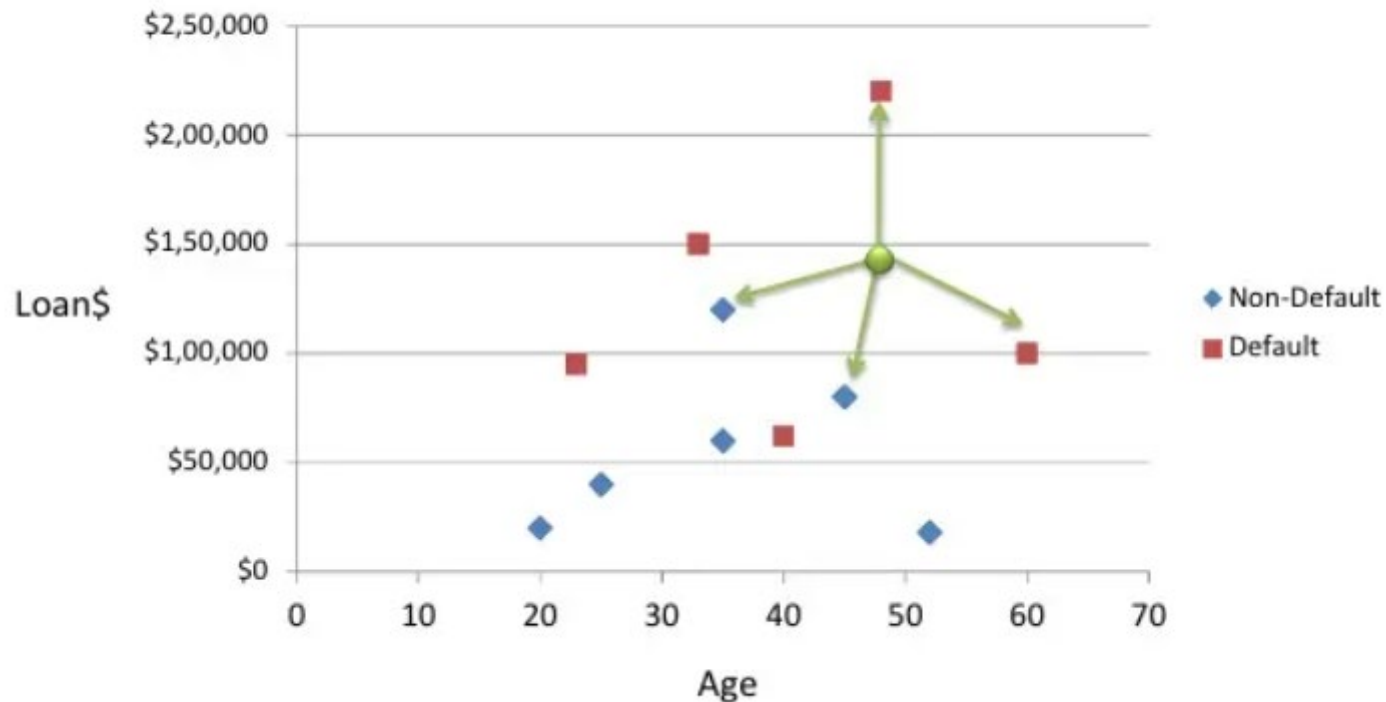
Nearest Neighbor Classification Algorithm

Let k be the number of nearest neighbors and D be the set of training examples.

1. **for** each test example $z = (\mathbf{x}', y')$ **do**
2. Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$
3. Select $D_z \subseteq D$, the set of k closest training examples to z .
4. $y' = \operatorname{argmax}_v \sum_{(x_i, y_i) \in D_z} I(v = y_i)$
5. **end for**



KNN Classification Example



Nearest-Neighbor Classifiers: Issues

Nearest-Neighbor Classifiers: Issues

- **Choosing the value of k**

Selecting the right number of neighbors is important. A very small or very large value of k can reduce accuracy.

- **Choosing the distance measure**

The result depends on how distance is calculated (for example, Euclidean or Manhattan distance).

- **Computational cost**

KNN is slow when the training dataset is large because distances must be calculated for many points.

- **High-dimensional data**

When data has many features, distance calculations become less meaningful and performance can decrease.



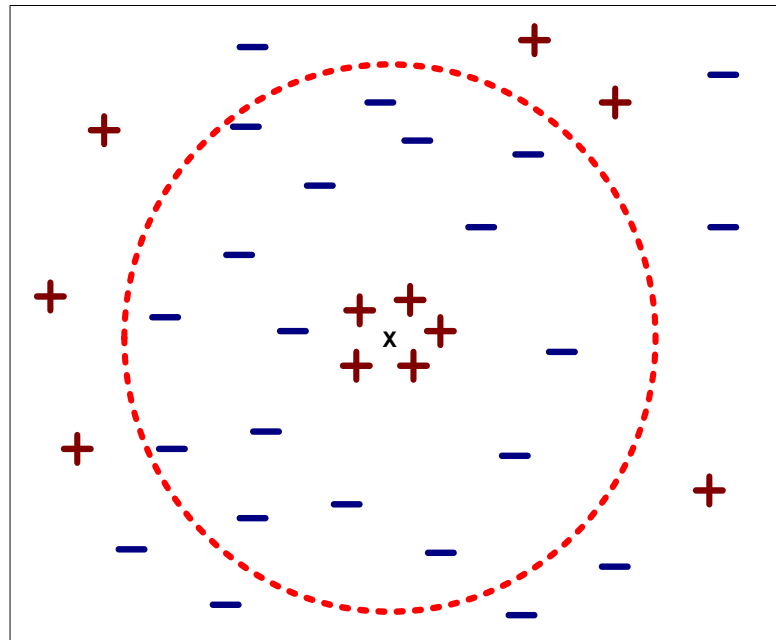
Value of K

- ▶ Choosing the value of k:
 - ▶ If k is too small, the classifier becomes very sensitive to noise, and a single wrong or unusual data point can affect the result.
 - ▶ If k is too large, the neighborhood may include data points from different classes, which can lead to wrong classification.

Rule of thumb:

$$K = \sqrt{N}$$

N: number of training points



Distance Metrics

Minkowsky:

$$D(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^m |x_i - y_i|^r \right)^{1/r}$$

Euclidean:

$$D(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

Manhattan / city-block:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m |x_i - y_i|$$

Camberra:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \frac{|x_i - y_i|}{|x_i + y_i|}$$

Chebychev:

$$D(\mathbf{x}, \mathbf{y}) = \max_{i=1}^m |x_i - y_i|$$

Quadratic:

$$D(\mathbf{x}, \mathbf{y}) = (\mathbf{x} - \mathbf{y})^T Q (\mathbf{x} - \mathbf{y}) = \sum_{j=1}^m \left(\sum_{i=1}^m (x_i - y_i) q_{ji} \right) (x_j - y_j)$$

Q is a problem-specific positive definite $m \times m$ weight matrix

Mahalanobis:

$$D(\mathbf{x}, \mathbf{y}) = [\det V]^{1/m} (\mathbf{x} - \mathbf{y})^T V^{-1} (\mathbf{x} - \mathbf{y})$$

V is the covariance matrix of $A_1..A_m$, and A_j is the vector of values for attribute j occurring in the training set instances 1.. n .

Correlation:

$$D(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^m (x_i - \bar{x}_i)(y_i - \bar{y}_i)}{\sqrt{\sum_{i=1}^m (x_i - \bar{x}_i)^2 \sum_{i=1}^m (y_i - \bar{y}_i)^2}}$$

$\bar{x}_i = \bar{y}_i$ and is the average value for attribute i occurring in the training set.

Chi-square:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \frac{1}{sum_i} \left(\frac{x_i}{size_x} - \frac{y_i}{size_y} \right)^2$$

sum_i is the sum of all values for attribute i occurring in the training set, and $size_x$ is the sum of all values in the vector $\mathbf{x}$.

Kendall's Rank Correlation:

$$D(\mathbf{x}, \mathbf{y}) = 1 - \frac{2}{n(n-1)} \sum_{i=1}^m \sum_{j=1}^{i-1} \text{sign}(x_i - x_j) \text{sign}(y_i - y_j)$$

$\text{sign}(x) = -1, 0$ or 1 if $x < 0$, $x = 0$, or $x > 0$, respectively.

Figure 1. Equations of selected distance functions.
($\mathbf{x}$ and $\mathbf{y}$ are vectors of m attribute values).

Distance Measure: Scale Effects

Effect of Different Feature Scales

Different features can have very different scales.

For example:

- Patient weight measured in kilograms (50–200)

- Blood protein level measured in ng/dL (−3 to 3)

Because of this, features with larger numeric ranges (like weight) have more influence on distance calculations.

This can bias the classifier and lead to incorrect results if features are not properly scaled.



Standardization

- ▶ Transform raw feature values into z-scores

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

- ▶ x_{ij} is the value for the i^{th} sample and j^{th} feature
 - ▶ μ_j is the average of all x_{ij} for feature j
 - ▶ σ_j is the standard deviation of all x_{ij} over all input samples
-
- ▶ Range and scale of z-scores should be similar (providing distributions of raw feature values are alike)
-

Nearest Neighbor : Dimensionality

- ▶ Problem with Euclidean measure:
 - ▶ High dimensional data
 - ▶ **curse of dimensionality**
 - ▶ Can produce counter-intuitive results
 - ▶ Shrinking density – sparsification effect

1	1	1	1	1	1	1	1	1	1	1	0
---	---	---	---	---	---	---	---	---	---	---	---

vs

1	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---

0	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---

0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---

$d = 1.4142$

$d = 1.4142$



Distance for Nominal Attributes

Value Difference Metric (VDM)

[Stanfill & Waltz, 1986]

Providing appropriate distance measurements for nominal attributes.

$$vdm_a(x, y) = \sum_{c=1}^C \left(\frac{N_{a,x,c}}{N_{a,x}} - \frac{N_{a,y,c}}{N_{a,y}} \right)^2$$

$N_{a,x}$ = # times attribute a had value x

$N_{a,x,c}$ = # times attribute a had value x and class was c

C = # output classes

Two values are considered closer
if they have more similar classifications, i.e.,
if they have more similar correlations with
the output classes.

Distance for Nominal Attributes

Suppose we have an attribute **Color** with values {Red, Blue, Green} and class labels {Positive, Negative}.

Color	#Positive	#Negative
Red	40	10
Blue	30	30
Green	10	40

Then:

$$P(\text{Positive}|\text{Red}) = 0.8, \quad P(\text{Negative}|\text{Red}) = 0.2$$

$$P(\text{Positive}|\text{Green}) = 0.2, \quad P(\text{Negative}|\text{Green}) = 0.8$$

So,

$$VDM(\text{Red}, \text{Green}) = |0.8 - 0.2| + |0.2 - 0.8| = 1.2$$

Whereas,

$$VDM(\text{Red}, \text{Blue}) = |0.8 - 0.5| + |0.2 - 0.5| = 0.6$$

Hence, **Red and Blue** are *more similar* than **Red and Green** — intuitively matching class behavior.



Distance for Heterogeneous Data

In this section, we define a heterogeneous distance function *HVDM* that returns the distance between two input vectors $\mathbf{x}$ and $\mathbf{y}$. It is defined as follows:

$$HVDM(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{a=1}^m d_a^2(x_a, y_a)} \quad (11)$$

where m is the number of attributes. The function $d_a(x, y)$ returns a distance between the two values x and y for attribute a and is defined as:

$$d_a(x, y) = \begin{cases} 1, & \text{if } x \text{ or } y \text{ is unknown; otherwise...} \\ normalized_vdm_a(x, y), & \text{if } a \text{ is nominal} \\ normalized_diff_a(x, y), & \text{if } a \text{ is linear} \end{cases} \quad (12)$$

Wilson, D. R. and Martinez, T. R., Improved Heterogeneous Distance Functions, Journal of Artificial Intelligence Research, vol. 6, no. 1, pp. 1-34, 1997



Distance for Heterogeneous Data

◆ Why we need it

Real-world datasets often look like this 🙋

Instance	Age	Color	Salary	Class
1	25	Red	40k	Yes
2	40	Blue	60k	No
3	30	Green	55k	Yes

Here:

- **Age** and **Salary** are numeric.
- **Color** is categorical.

🙋 Euclidean distance fails (can't subtract "Red" – "Blue").

🙋 VDM only handles categorical attributes.

So, **HVDM** combines both in a consistent way.

Distance for Heterogeneous Data

◆ Formula

The HVDM distance between two instances x and y is:

$$HVDM(x, y) = \sqrt{\sum_{a=1}^m d_a(x_a, y_a)^2}$$

where m = number of attributes, and

$$d_a(x_a, y_a) = \begin{cases} \frac{|x_a - y_a|}{4 \times \sigma_a}, & \text{if attribute } a \text{ is numeric} \\ \sqrt{VDM_a(x_a, y_a)}, & \text{if attribute } a \text{ is categorical} \end{cases}$$

✚ Explanation of parts

- **Numeric part:**

Each numeric feature is scaled by its **standard deviation** (σ_a), and divided by 4 to normalize it to roughly the same range (based on $\pm 2\sigma$ rule).

This avoids one numeric feature dominating the others.

- **Categorical part:**

$VDM_a(x_a, y_a)$ measures the dissimilarity between two categorical values (as we discussed earlier).

Distance for Heterogeneous Data

Dataset

Instance	Age	Color	Class
1	20	Red	Positive
2	40	Blue	Negative
3	25	Red	Positive
4	35	Green	Negative

Assume we want to find the distance between Instance 1 and Instance 2.

Step 1: Compute numeric feature difference (Age)

$$d_{Age}(1, 2) = \frac{|20 - 40|}{4 \times \sigma_{Age}}$$

Let's say the standard deviation of Age, $\sigma_{Age} = 8.16$.

Then:

$$d_{Age}(1, 2) = \frac{20}{4 \times 8.16} = \frac{20}{32.64} \approx 0.613$$

Distance for Heterogeneous Data

Dataset

Instance	Age	Color	Class
1	20	Red	Positive
2	40	Blue	Negative
3	25	Red	Positive
4	35	Green	Negative

Assume we want to find the distance between Instance 1 and Instance 2.

Step 2: Compute categorical feature difference (Color)

From the data:

Color	P(Positive)	P(Negative)
Red	1.0	0.0
Blue	0.0	1.0
Green	0.0	1.0

$$VDM(Red, Blue) = |1.0 - 0.0| + |0.0 - 1.0| = 2$$

$$d_{Color}(Red, Blue) = \sqrt{VDM(Red, Blue)} = \sqrt{2} = 1.414$$

Distance for Heterogeneous Data

Dataset

Instance	Age	Color	Class
1	20	Red	Positive
2	40	Blue	Negative
3	25	Red	Positive
4	35	Green	Negative

Assume we want to find the distance between Instance 1 and Instance 2.

Step 3: Combine them using HVDM

$$HVDM(1, 2) = \sqrt{d_{Age}^2 + d_{Color}^2} = \sqrt{(0.613)^2 + (1.414)^2} = \sqrt{0.376 + 2.000} = \sqrt{2.376} \approx 1.54$$

✅ Final distance = 1.54



Nearest Neighbour : Computational Complexity

► Expensive

- To determine the nearest neighbour of a query point q , must compute the distance to all N training examples
 - + Pre-sort training examples into fast data structures (kd-trees)
 - + Compute only an approximate distance (LSH)
 - + Remove redundant data (condensing)

► Storage Requirements

- Must store all training data **P**
 - + Remove redundant data (condensing)
 - Pre-sorting often increases the storage requirements

► High Dimensional Data

- “Curse of Dimensionality”
 - Required amount of training data increases exponentially with dimension
 - Computational cost also increases dramatically
 - Partitioning techniques degrade to linear search in high dimension

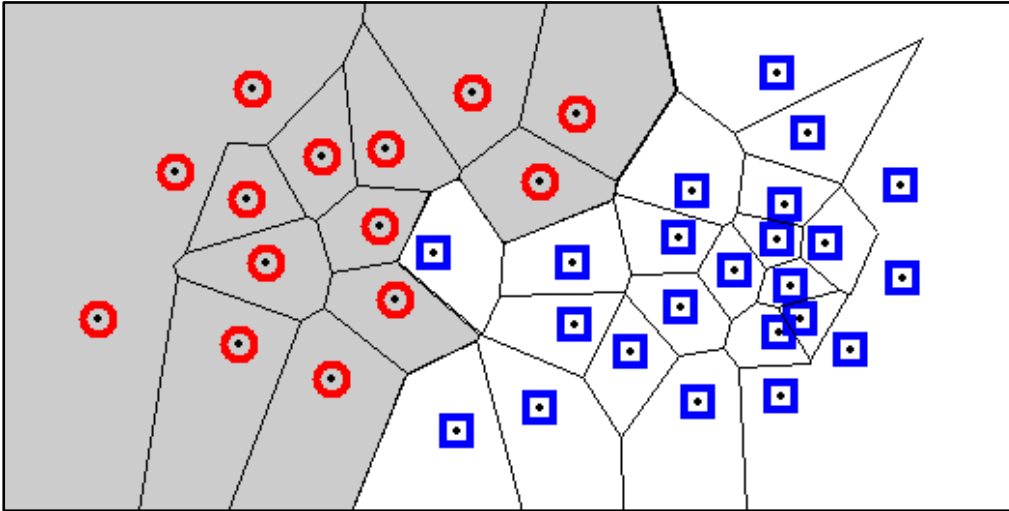


Reduction in Computational Complexity

- ▶ Reduce size of training set
 - ▶ Condensation, editing
- ▶ Use geometric data structure for high dimensional search



Condensation: Decision Regions



Each cell contains one sample, and every location within the cell is closer to that sample than to any other sample.

A Voronoi diagram divides the space into such cells.

Every query point will be assigned the classification of the sample within that cell. The *decision boundary* separates the class regions based on the 1-NN decision rule.

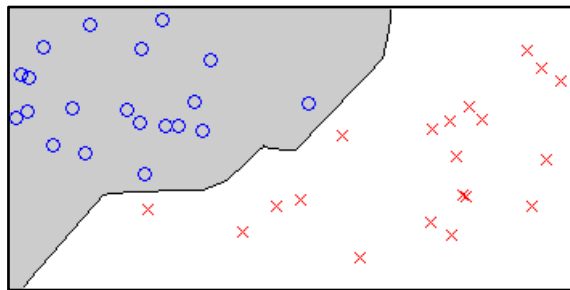
Knowledge of this boundary is sufficient to classify new points.

The boundary itself is rarely computed; many algorithms seek to retain only those points necessary to generate an identical boundary.

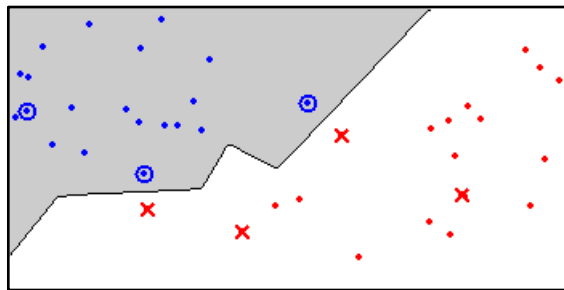


Condensing

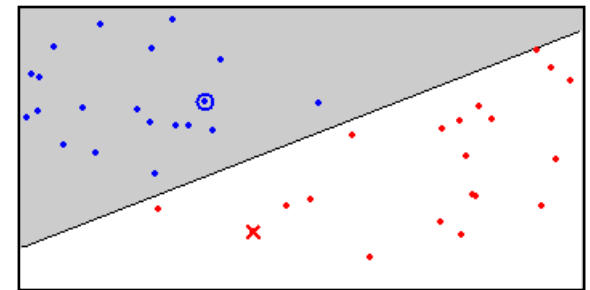
- ▶ Aim is to reduce the number of training samples
- ▶ Retain only the samples that are needed to define the decision boundary
- ▶ Decision Boundary Consistent – a subset whose nearest neighbour decision boundary is identical to the boundary of the entire training set
- ▶ Minimum Consistent Set – the smallest subset of the training data that correctly classifies all of the original training data



Original data



Condensed data



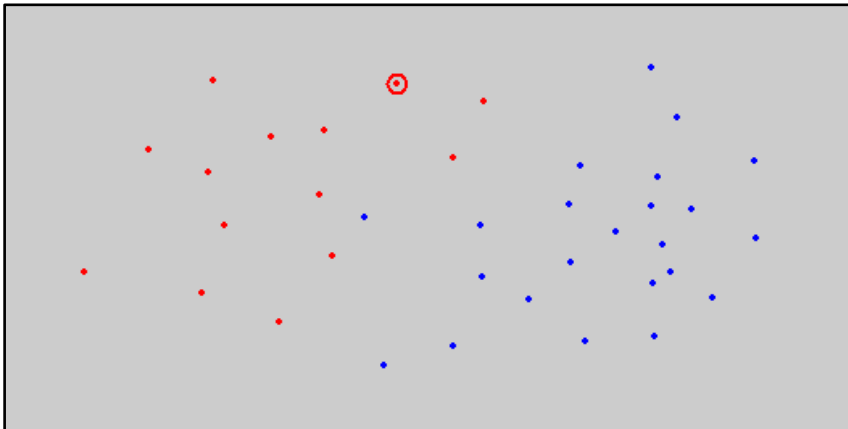
Minimum Consistent Set

Condensing

► Condensed Nearest Neighbour (CNN)

1. Initialize subset with a single (or K) training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full

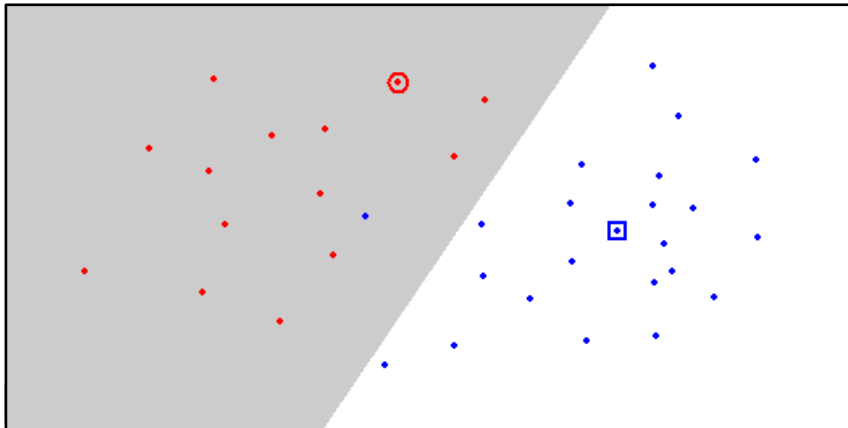
- Incremental
- Order dependent
- Neither minimal nor decision boundary consistent
- $O(n^3)$ for brute-force method



Condensing

► Condensed Nearest Neighbour (CNN)

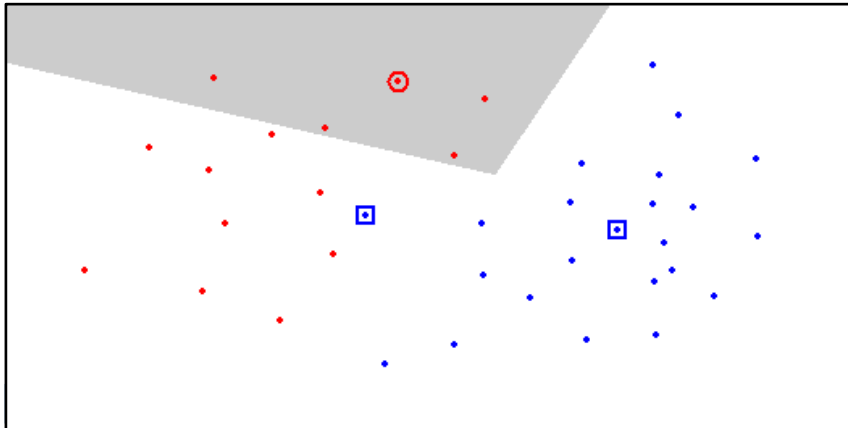
1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



Condensing

► Condensed Nearest Neighbour (CNN)

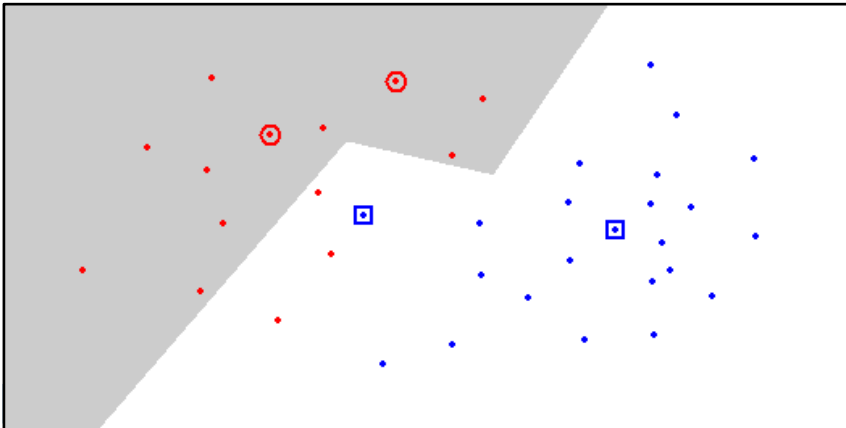
1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



Condensing

► Condensed Nearest Neighbour (CNN)

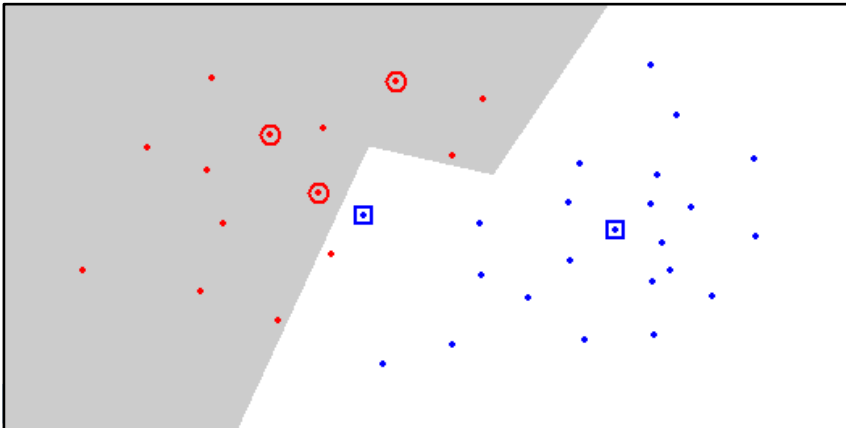
1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



Condensing

► Condensed Nearest Neighbour (CNN)

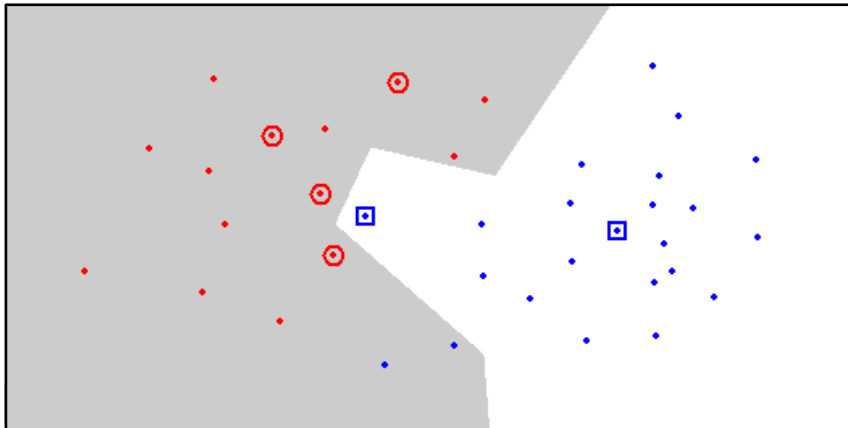
1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



Condensing

► Condensed Nearest Neighbour (CNN)

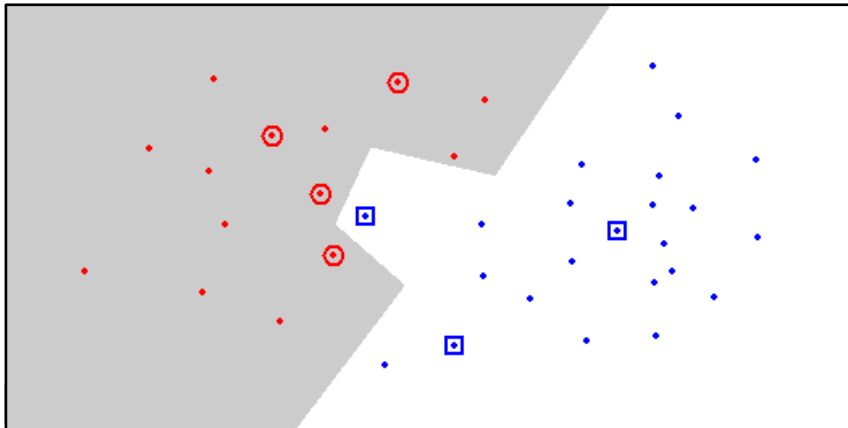
1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



Condensing

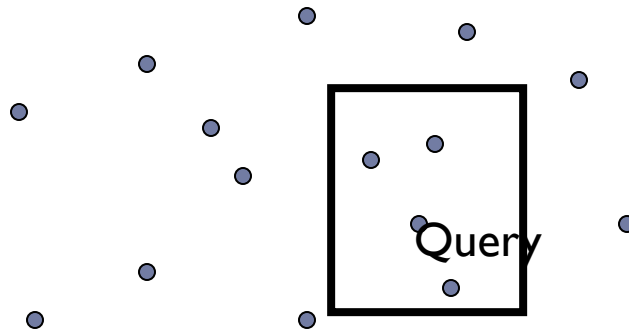
► Condensed Nearest Neighbour (CNN)

1. Initialize subset with a single training example
2. Classify all remaining samples using the subset, and transfer any incorrectly classified samples to the subset
3. Return to 2 until no transfers occurred or the subset is full



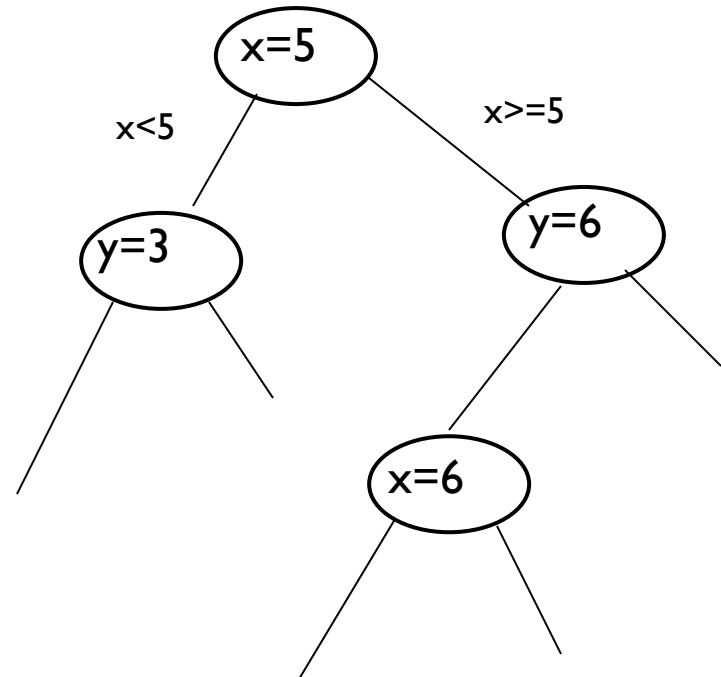
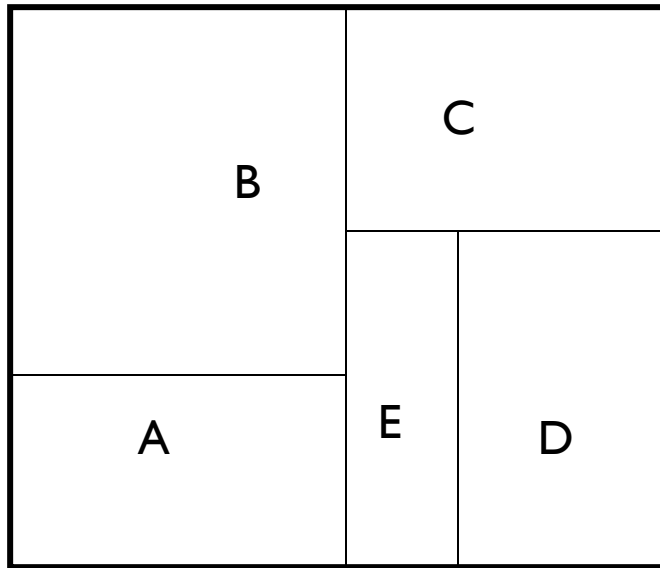
High dimensional search

- ▶ Given a point set and a nearest neighbor query point
- ▶ Find the points enclosed in a rectangle (range) around the query
- ▶ Perform linear search for nearest neighbor only in the rectangle

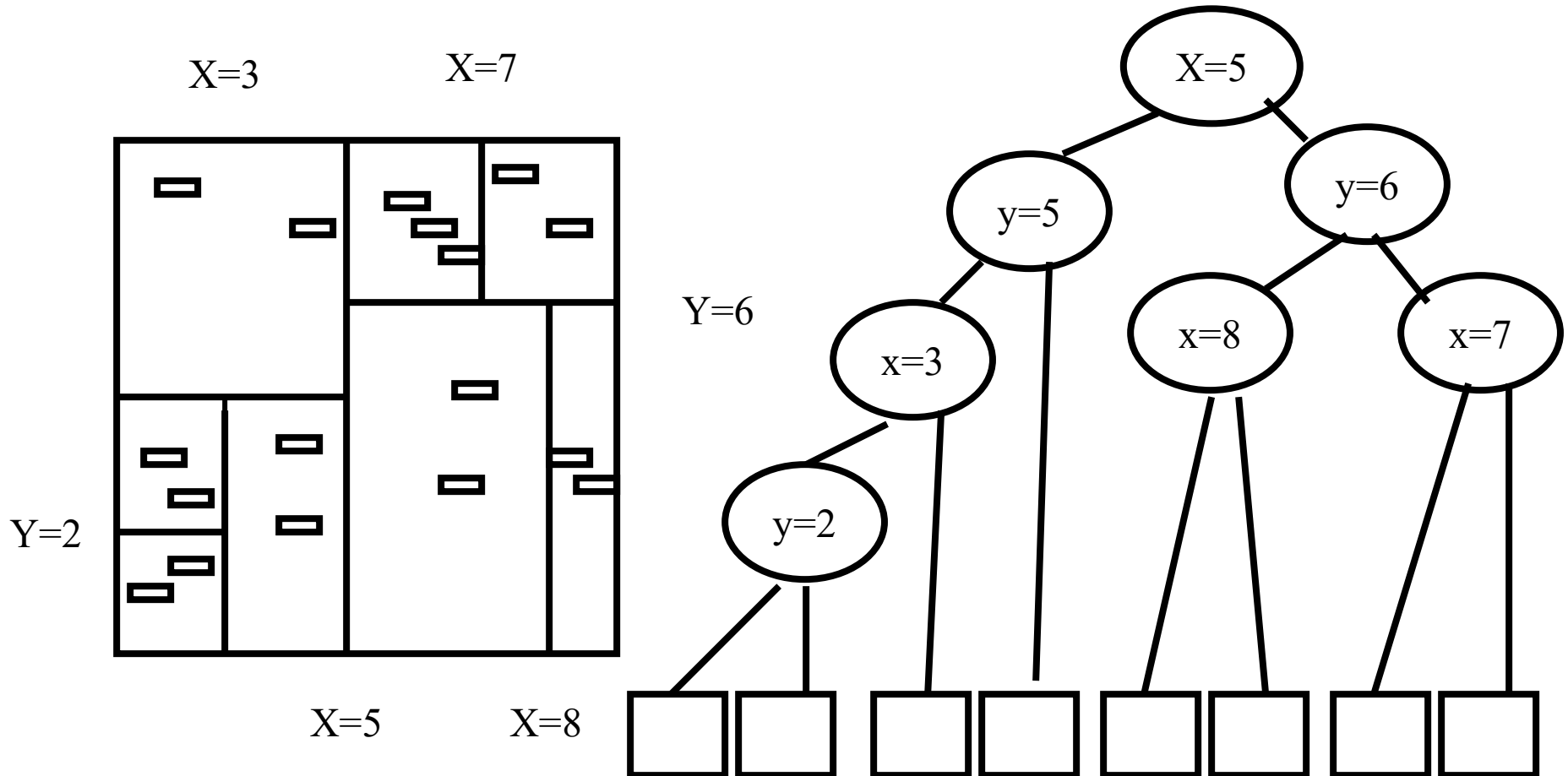


kd-tree: data structure for range search

- ▶ Index data into a tree
- ▶ Search on the tree
- ▶ Tree construction: At each level we use a different dimension to split



kd-tree example



KNN: Alternate Terminologies

KNN: Alternate Terminologies

KNN is known by different names because of how it works:

- **Instance-Based Learning**

KNN makes decisions by using **stored training examples** directly, instead of learning a model.

- **Lazy Learning**

KNN does **not train a model in advance**. It waits until a query is given and then performs computation.

- **Case-Based Reasoning**

KNN solves new problems by **comparing them with similar past cases**.

- **Exemplar-Based Learning**

KNN uses **representative examples (exemplars)** from the dataset to make predictions.

